LAB WORK 2

Database Systems — Relational Model & Keys

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee

1. Six different superkeys

Assuming EmpID, SSN, Email, and Phone are unique:

1. {EmpID}
2. {SSN}
3. {Email}
4. {Phone}
5. {EmpID, Name}
6. {SSN, Department}

2. Candidate keys

Candidate keys are minimal superkeys:

- EmpID
- SSN
- Email
- Phone

These attributes can uniquely identify an employee, assuming each is required to be unique.

3. Primary key choice

I would choose EmpID as the primary key because it is a stable, system-generated identifier. It does not depend on personal information and is convenient for foreign key relationships.

4. Can two employees have the same phone number?

Under the assumption that Phone is a candidate key, two employees cannot have the same phone number. However, the sample data alone does not prove that phone numbers must be unique. The database rules must explicitly enforce this constraint.

Relation B: Course Registration

1. Minimum attributes for the primary key

The primary key is:

(StudentID, CourseCode, Section, Semester, Year)

2. Why each attribute is necessary

- StudentID identifies the student.
- CourseCode identifies the course.
- Section identifies a particular section of the course.
- Semester identifies the academic semester.
- Year distinguishes academic years.

Together, these attributes identify a student's registration in a particular course section during a specific semester.

3. Additional candidate keys

No additional candidate keys can be established from the given business rules. If the university assigns a unique RegistrationID, it could serve as an additional candidate key.

Credits and Grade are not part of the primary key because they describe the registration rather than identify it.

Task 1.2: Foreign Key Design

The foreign key relationships are:

1. Student.AdvisorID → Professor.ProfID
2. Student.Major → Department.DeptCode
3. Course.DepartmentCode → Department.DeptCode
4. Department.ChairID → Professor.ProfID
5. Enrollment.StudentID → Student.StudentID
6. Enrollment.CourseID → Course.CourseID

The primary key of Enrollment is (StudentID, CourseID, Semester), assuming a student can take a course only once per semester.

AdvisorID and ChairID may be nullable if a student has no assigned advisor or a department has no assigned chair.

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System

1. Entities

Strong entities:

- Patient
- Doctor
- Department
- Appointment

- Prescription

Weak or dependent entities:

- PatientPhone
- DoctorPhone
- DoctorSpecialization
- HospitalRoom

HospitalRoom is weak because its room number is unique only within a department.

2. Attributes

Patient

- PatientID (Primary Key)
- Name
- BirthDate
- Address (Street, City, State, ZIP)
- InsuranceInformation
- PhoneNumber (multivalued)

Doctor

- DoctorID (Primary Key)
- Name
- Specialization (multivalued)
- PhoneNumber (multivalued)
- OfficeLocation

Department

- DeptCode (Primary Key)
- DeptName
- Location

Appointment

- AppointmentID (Primary Key)
- AppointmentDateTime
- Purpose
- Notes

Prescription

- PrescriptionID (Primary Key)
- Medication
- Dosage
- Instructions

HospitalRoom

- DeptCode (Partial Key component)
- RoomNumber (Partial Key component)
- RoomType

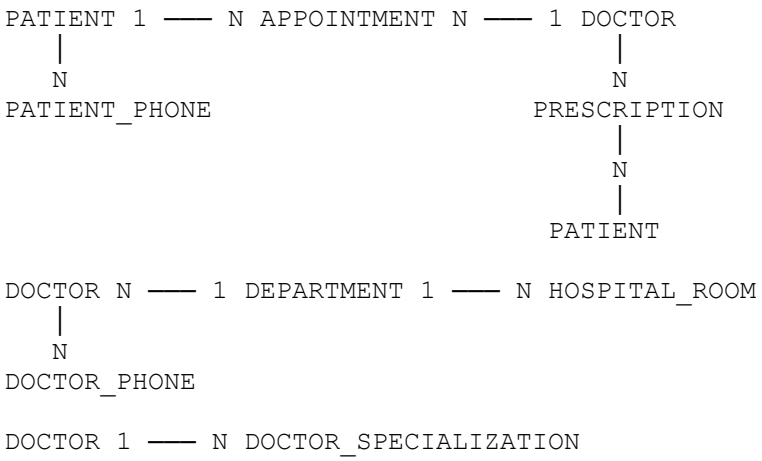
The primary key of HospitalRoom is (DeptCode, RoomNumber).

3. Relationships and cardinalities

- Patient — Appointment: 1
- Doctor — Appointment: 1
- Doctor — Department: N:1, assuming each doctor belongs to one department
- Department — HospitalRoom: 1
- Doctor — Prescription: 1
- Patient — Prescription: 1
- Appointment — Prescription: 1, if prescriptions are linked to appointments
- Patient — PatientPhone: 1
- Doctor — DoctorPhone: 1
- Doctor — DoctorSpecialization: 1

4. ER diagram structure

Create the following entities in ERDPlus and connect them using the relationships above. Use double rectangles for weak entities and double diamonds for identifying relationships if using Chen notation.



5. Primary keys

- Patient: PatientID
- Doctor: DoctorID
- Department: DeptCode
- Appointment: AppointmentID
- Prescription: PrescriptionID
- HospitalRoom: (DeptCode, RoomNumber)
- PatientPhone: (PatientID, PhoneNumber)
- DoctorPhone: (DoctorID, PhoneNumber)
- DoctorSpecialization: (DoctorID, Specialization)

Task 2.2: E-commerce Platform

1. Entities and attributes

Customer

- CustomerID (PK)
- Name
- Email
- BillingAddress

CustomerAddress

- AddressID (PK)
- CustomerID (FK)
- Street
- City
- PostalCode
- AddressType

Orders

- OrderID (PK)
- CustomerID (FK)
- OrderDate
- ShippingAddressID (FK)
- TotalAmount

Product

- ProductID (PK)
- ProductName
- Price
- CategoryID (FK)
- VendorID (FK)

Category

- CategoryID (PK)
- CategoryName

Vendor

- VendorID (PK)
- VendorName
- ContactInformation

OrderItem

- OrderID (PK, FK)
- ProductID (PK, FK)
- Quantity
- PriceAtOrder

Review

- ReviewID (PK)
- CustomerID (FK)
- ProductID (FK)
- Rating
- ReviewText
- ReviewDate

Inventory

- ProductID (PK, FK)
- QuantityInStock
- ReorderLevel

2. Relationships

- Customer — Orders: 1
- Customer — CustomerAddress: 1
- CustomerAddress — Orders: 1
- Category — Product: 1
- Vendor — Product: 1
- Orders — OrderItem: 1
- Product — OrderItem: 1
- Customer — Review: 1
- Product — Review: 1
- Product — Inventory: 1:1

3. Weak entity

OrderItem is a weak or existence-dependent entity because it cannot exist without an order and a product. Its composite primary key is (OrderID, ProductID).

4. Many-to-many relationship with attributes

Orders and Products have an M relationship. OrderItem resolves this relationship and stores Quantity and PriceAtOrder.

PriceAtOrder is necessary because the current product price may differ from the price paid when the order was placed.

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

1. Functional dependencies

Assume each student has one major, each project has one title and type, and each supervisor has one name and department.

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID) → Role, HoursWorked

The primary key is (StudentID, ProjectID), assuming each student has one role per project.

2. Problems in the table

Redundancy: Student information is repeated for every project. Project information and supervisor details are repeated for every student working on the same project.

Update anomaly: If a supervisor changes departments, multiple rows must be updated.

Insert anomaly: A new project cannot be stored independently if no student has been assigned to it.

Delete anomaly: Deleting the only student assigned to a project may also remove the only record of that project.

3. First Normal Form (1NF)

The table satisfies 1NF if every attribute contains a single atomic value and there are no repeating groups.

To maintain 1NF, store one student-project assignment per row and avoid lists of students, roles, or hours inside a single cell.

4. Second Normal Form (2NF)

The primary key is (StudentID, ProjectID).

Partial dependencies:

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

These attributes depend on only part of the composite key.

2NF decomposition:

Student(StudentID PK, StudentName, StudentMajor)

Project(ProjectID PK, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate, SupervisorName, SupervisorDept)

StudentProject(StudentID PK/FK, ProjectID PK/FK, Role, HoursWorked)

5. Third Normal Form (3NF)

A transitive dependency exists:

ProjectID → SupervisorID → SupervisorName, SupervisorDept

To remove it, create a separate Supervisor table.

Final 3NF decomposition:

Student(
StudentID PK,
StudentName,
StudentMajor
)

Supervisor(
SupervisorID PK,
SupervisorName,
SupervisorDept
)

Project(
ProjectID PK,
ProjectTitle,
ProjectType,
SupervisorID FK,
StartDate,
EndDate
)

StudentProject(
StudentID PK/FK,
ProjectID PK/FK,
Role,
HoursWorked
)

The final design reduces redundancy and prevents update, insertion, and deletion anomalies.

Task 4.2: Advanced Normalization

1. Primary key

The primary key is (StudentID, CourseID), assuming each student can enroll in only one section of a given course. If a course can have multiple sections, a SectionID must be added to the schema and key.

2. Functional dependencies

- StudentID → StudentMajor
- CourseID → CourseName
- InstructorID → InstructorName
- Room → Building
- CourseID → InstructorID, TimeSlot, Room, assuming each course has only one section

- (StudentID, CourseID) identifies a student-course enrollment

If courses have multiple sections, the dependency describing the instructor, time slot, and room must instead use SectionID or another section identifier.

3. BCNF check

The table is not in BCNF because StudentID, CourseID, InstructorID, and Room determine attributes but are not all superkeys of the original table.

For example, StudentID → StudentMajor, but StudentID alone does not identify a complete row.

4. BCNF decomposition

Student

Student(
StudentID PK,
StudentMajor
)

Course

Course(
CourseID PK,
CourseName
)

Instructor

Instructor(
InstructorID PK,
InstructorName
)

Room

Room(
Room PK,
Building
)

CourseSection

CourseSection(
CourseID PK,
InstructorID FK,
TimeSlot,
Room FK
)

Enrollment

Enrollment(
StudentID PK/FK,
CourseID PK/FK
)

If multiple sections exist for the same course, use CourseSection(SectionID PK, CourseID FK, InstructorID FK, TimeSlot, Room FK) and Enrollment(StudentID PK/FK, SectionID PK/FK).

5. Potential loss of information

A decomposition can lose information if the original relationships between students, courses, instructors, times, and rooms cannot be reconstructed through joins.

The proposed decomposition is lossless under the stated functional dependencies and the assumption of one section per course. When courses have multiple sections, using SectionID in CourseSection and Enrollment preserves the exact enrollment-to-section relationship.

Part 5: Design Challenge

Task 5.1: University Clubs and Organizations

1. Entities and attributes

Student

- StudentID (PK)
- StudentName
- Email

Club

- ClubID (PK)
- ClubName
- Description
- Budget
- AdvisorID (FK)

FacultyAdvisor

- AdvisorID (PK)
- AdvisorName
- Department

Membership

- StudentID (PK, FK)
- ClubID (PK, FK)
- JoinDate
- MembershipStatus

OfficerPosition

- PositionID (PK)
- PositionName

ClubOfficer

- StudentID (PK, FK)
- ClubID (PK, FK)
- PositionID (FK)
- StartDate
- EndDate

ClubEvent

- EventID (PK)
- ClubID (FK)
- EventName
- EventDateTime
- Description
- RoomID (FK)

Attendance

- EventID (PK, FK)
- StudentID (PK, FK)
- AttendanceStatus

Room

- RoomID (PK)
- Building
- RoomNumber
- Capacity

Expense

- ExpenseID (PK)
- ClubID (FK)
- ExpenseDate
- Amount
- Description

RoomReservation

- ReservationID (PK)
- RoomID (FK)
- EventID (FK)
- StartTime
- EndTime

2. Relationships

- Student — Membership — Club: M
- FacultyAdvisor — Club: 1
- Club — ClubEvent: 1
- ClubEvent — Attendance — Student: M
- Club — ClubOfficer: 1
- Student — ClubOfficer: 1
- OfficerPosition — ClubOfficer: 1
- Room — RoomReservation: 1
- ClubEvent — RoomReservation: 1:1, assuming each event has at most one reservation
- Club — Expense: 1

3. Normalized relational schema

Student(StudentID PK, StudentName, Email)

FacultyAdvisor(AdvisorID PK, AdvisorName, Department)

Club(ClubID PK, ClubName, Description, Budget, AdvisorID FK)

Membership(StudentID PK/FK, ClubID PK/FK, JoinDate, MembershipStatus)

OfficerPosition(PositionID PK, PositionName)

ClubOfficer(StudentID PK, ClubID PK, PositionID FK, StartDate, EndDate, FK(StudentID, ClubID) → Membership)

ClubEvent(EventID PK, ClubID FK, EventName, EventDateTime, Description)

Room(RoomID PK, Building, RoomNumber, Capacity)

RoomReservation(ReservationID PK, RoomID FK, EventID FK UNIQUE, StartTime, EndTime)

Attendance(EventID PK/FK, StudentID PK/FK, AttendanceStatus)

Expense(ExpenseID PK, ClubID FK, ExpenseDate, Amount, Description)

4. Design decision

I chose a separate Membership table to represent the many-to-many relationship between students and